

ESCUELA SUPERIOR POLITÉCNICA DEL LITORAL

Materia: Sistemas Embebidos

Tarea°2: COMUNICACIÓN ENTRE MICROCONTROLADORES.

Paralelo N°: 1

Grupo N°: 5

Integrantes:

Dilan Matthew Chele Marcillo

Fernando David Casierra Almeida.

Fabricio Jacob Barcia Matias.

Joao Paulo Pincay Sanabria.

Docente: Msc Ronald Solís.

Fecha de Entrega: 10/Jun/2026

Término Académico: 1 PAO 2026-2027

Guayaquil – Ecuador

Objetivo general:

Diseñar e implementar un sistema embebido basado en la comunicación entre microcontroladores que permita ejecutar un juego de memoria interactivo mediante una matriz LED 8×8, pulsadores y generación de efectos de sonido.

Objetivo específico:

- Implementar la lógica del juego utilizando un ATmega328P.
- Establecer comunicación entre el ATmega328P y el PIC16F887.
- Mostrar secuencias visuales mediante una matriz LED 8×8.
- Reproducir efectos de sonido de acuerdo con los eventos del juego.

Descripción técnica del juego:

El proyecto consiste en un juego de memoria inspirado en el clásico Simon Says. Al iniciar el sistema, se muestra un mensaje de bienvenida en la matriz LED 8×8 y el usuario debe seleccionar una dificultad. Dependiendo de la opción elegida, la velocidad con la que se muestran las secuencias será diferente, siendo la dificultad difícil la más rápida.

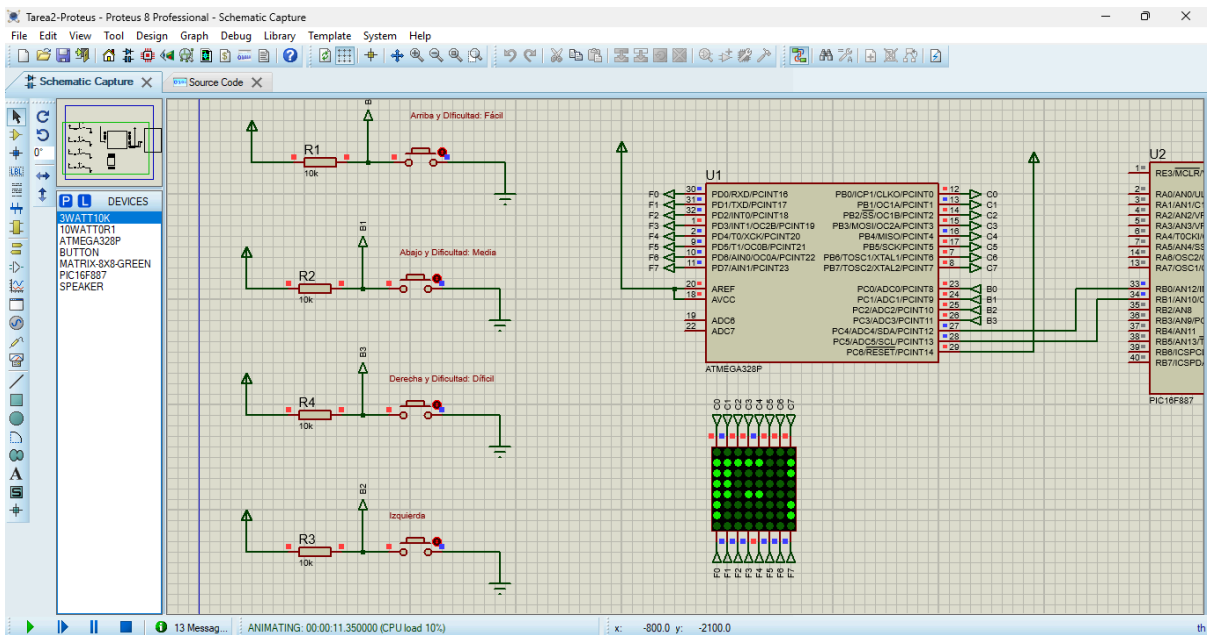
Una vez seleccionada la dificultad, el ATmega328P genera una secuencia aleatoria de movimientos compuesta por flechas que representan las direcciones arriba, abajo, izquierda y derecha. Estas flechas son mostradas al usuario mediante la matriz LED 8×8 durante un tiempo determinado por la dificultad seleccionada.

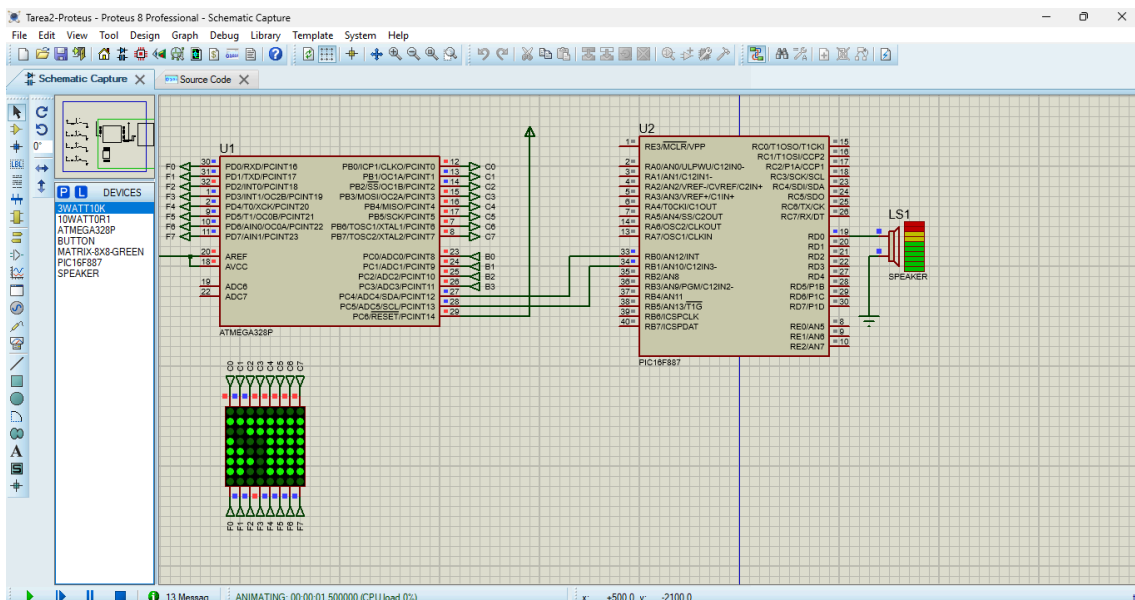
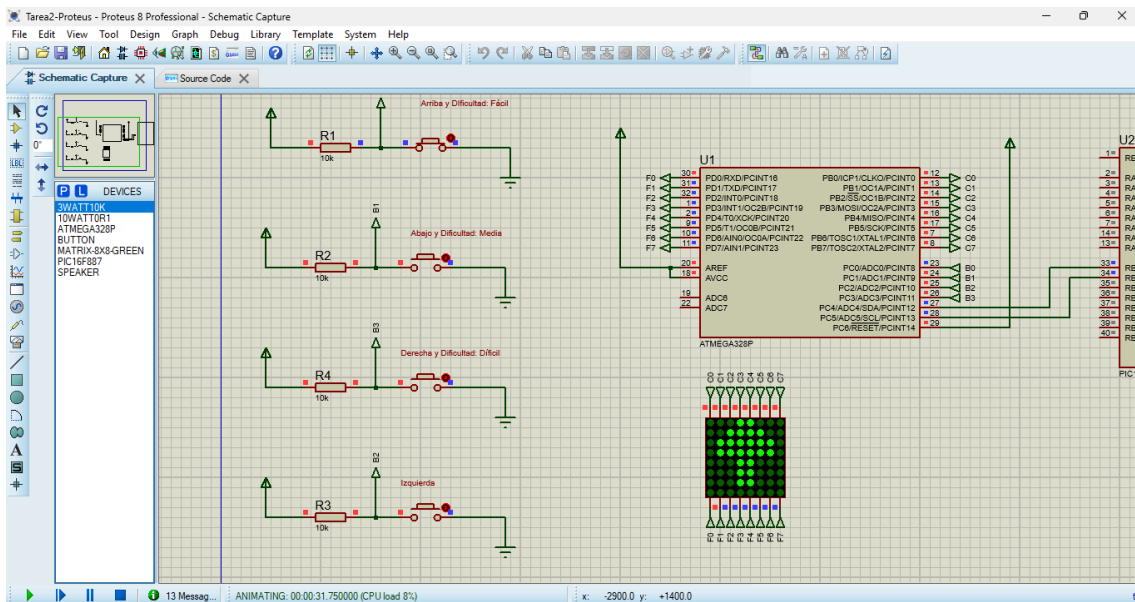
Después de observar la secuencia, el usuario debe reproducirla utilizando cuatro pulsadores, donde cada botón corresponde a una dirección específica. Cada pulsación es registrada por el microcontrolador y almacenada temporalmente para su posterior comparación con la secuencia original.

Al finalizar el ingreso de datos, el sistema verifica si la secuencia introducida coincide con la generada. Si la respuesta es correcta, se muestra un símbolo de acierto, se reproduce un sonido de confirmación y el jugador avanza a la siguiente ronda, en la que deberá recordar una secuencia más larga. Si la respuesta es incorrecta, se muestra un mensaje de error, se reproduce un sonido de derrota y el juego finaliza mostrando la pantalla de "Game Over".

El juego está compuesto por un máximo de seis rondas. Si el jugador logra completar correctamente todas las secuencias, se muestra el mensaje "Champion" junto con un sonido de victoria, indicando que ha superado exitosamente el desafío.

Simulación en Proteus:





Explicación del código:

ATmega328P

El programa comienza definiendo la frecuencia de trabajo del microcontrolador en 8 MHz e incluyendo las librerías necesarias para acceder a los registros del ATmega328P, generar retardos y utilizar funciones de generación de números aleatorios. Además, se asigna un valor numérico a cada dirección del juego: arriba, abajo, izquierda y derecha.

Se declaran los arreglos que contienen los patrones binarios utilizados para representar imágenes y textos en la matriz LED 8×8. Y se declaran las variables globales del programa. El arreglo secuencia almacena los movimientos que genera el sistema,

mientras que el arreglo respuesta guarda los movimientos ingresados por el usuario. También se utilizan variables para controlar la ronda actual, la dificultad seleccionada, la velocidad de visualización y el tiempo que cada flecha permanece visible en pantalla.

La función `enviar_sonido()` se encarga de la comunicación entre el ATmega328P y el PIC16F887. Cuando ocurre un evento importante en el juego, como una pulsación, una victoria o una derrota, esta función envía un comando mediante los pines PC4 y PC5. El PIC recibe esta información y genera el efecto de sonido correspondiente mediante el buzzer.

La función `generar_secuencia()` crea la secuencia aleatoria que el jugador deberá memorizar. Para ello genera seis números aleatorios entre 1 y 4, correspondientes a las cuatro direcciones posibles. Además, se implementa una verificación para evitar que dos movimientos consecutivos sean iguales, haciendo el juego más dinámico.

La función `mostrarMatriz()` es la responsable de controlar la matriz LED 8×8. Su trabajo consiste en activar rápidamente las filas y columnas correspondientes para formar imágenes o desplazar textos completos a través de la pantalla.

La función `pausa()` simplemente genera un retardo configurable en milisegundos. Se utiliza principalmente para controlar el tiempo entre la visualización de una flecha y la siguiente.

La función `seleccionar_dificultad()` espera a que el usuario presione uno de los botones disponibles. Dependiendo del botón seleccionado, se configura una velocidad distinta para el juego.

La función `mostrar_patron()` tiene la tarea de visualizar una imagen específica en la matriz LED, como una flecha o un símbolo de acierto o error. El tiempo durante el cual la imagen permanece visible depende de la dificultad seleccionada.

La función `mostrar_secuencia()` reproduce la secuencia que el jugador debe memorizar. Recorre el arreglo donde se almacenan los movimientos generados y muestra en pantalla la flecha correspondiente a cada dirección. Además, durante cada paso se envía una señal al PIC para reproducir un sonido de acompañamiento.

La función `verificar()` compara la secuencia original con la respuesta ingresada por el jugador. Si todos los movimientos coinciden, devuelve un valor indicando que la respuesta es correcta. Si encuentra al menos una diferencia, devuelve un valor de error que provoca la finalización de la ronda.

Finalmente, la función `main()` contiene la lógica principal del juego. En primer lugar configura los puertos del microcontrolador, estableciendo como salidas los pines conectados a la matriz LED y al sistema de sonido.

Después de la configuración inicial, el sistema muestra el mensaje de inicio, solicita la selección de dificultad y genera la secuencia aleatoria. A continuación entra en un ciclo infinito donde se desarrolla el juego. En cada ronda se muestra la secuencia

correspondiente, el usuario introduce su respuesta mediante los botones y el sistema verifica si los movimientos coinciden.

Si la respuesta es correcta, se muestra un símbolo de confirmación y el jugador avanza a la siguiente ronda. Cuando logra superar las seis rondas disponibles, aparece el mensaje "CHAMPION", se reproduce un sonido de victoria y el juego se reinicia desde el menú principal.

Por el contrario, si el usuario comete un error, se muestra un símbolo de fallo seguido del mensaje "GAME OVER". Se reproduce el sonido de derrota y el sistema vuelve al menú inicial para comenzar una nueva partida. De esta manera, el programa implementa completamente la lógica del juego de memoria y coordina la interacción entre la matriz LED, los botones y el sistema de sonido controlado por el PIC16F887.

PIC16F887

Al inicio del programa se desactivan los canales analógicos mediante los registros ANSEL y ANSELH para que todos los pines funcionen como entradas y salidas digitales. Luego se configura el puerto B como entrada, ya que por este puerto se reciben los comandos enviados por el ATmega328P. También se ajusta el oscilador interno del PIC a una frecuencia de 8 MHz y se inicializa la librería de sonido utilizando el pin RD0, donde se encuentra conectado el buzzer.

Dentro de un ciclo infinito, el PIC lee constantemente los pines RB0 y RB1. Estas dos líneas de comunicación permiten recibir comandos binarios enviados por el ATmega328P. El valor recibido se almacena en la variable comando y determina qué sonido debe reproducirse.

Cuando el comando recibido es igual a 1, el PIC genera un tono corto y agudo de 880 Hz durante 50 ms. Este sonido se utiliza para indicar acciones como la pulsación de botones, la selección de dificultad o la reproducción de una flecha durante la secuencia.

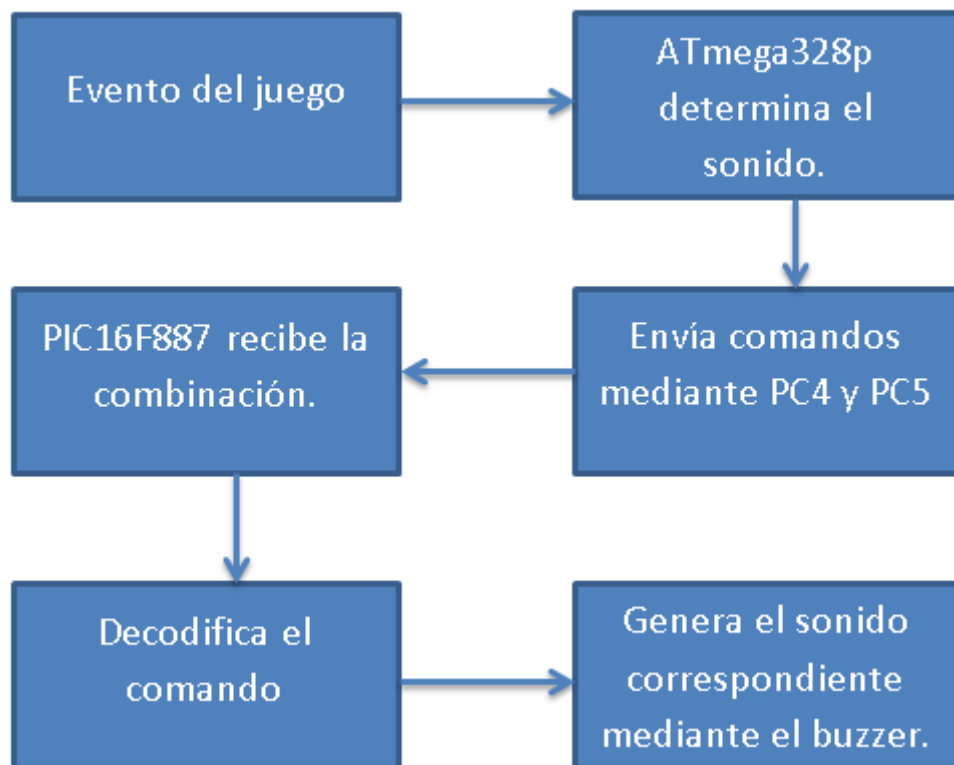
Cuando el comando recibido es igual a 2, el PIC reproduce una secuencia de notas descendentes. Esta melodía se utiliza cuando el jugador pierde la partida y aparece el mensaje "GAME OVER", generando un efecto de derrota.

Cuando el comando recibido es igual a 3, el PIC reproduce una melodía ascendente compuesta por varias notas musicales. Este sonido se utiliza tanto al iniciar el juego como cuando el jugador completa exitosamente todas las rondas y obtiene la victoria.

Descripción del esquema:

Funcionamiento:

El ATmega328P controla toda la lógica del juego. Cuando ocurre un evento (inicio, acierto o error), el ATmega envía una combinación lógica por los pines PC4 y PC5. El PIC16F887 monitorea continuamente esas entradas. Según la combinación recibida, ejecuta un sonido diferente en el buzzer. El PIC no participa en la lógica del juego; únicamente genera los efectos de audio.



Enlace del repositorio de GitHub: [Tarea2-Grupo5/ATmega-328P at main · Fabricio0011/Tarea2-Grupo5](https://github.com/Tarea2-Grupo5/ATmega-328P-at-main-Fabricio0011/Tarea2-Grupo5)

Conclusiones:

La comunicación entre el ATmega328P y el PIC16F887 permitió distribuir las tareas de procesamiento y generación de sonido.

La simulación en Proteus permitió validar el correcto funcionamiento del sistema antes de una implementación física.

Recomendaciones:

Incorporar almacenamiento de puntuaciones utilizando memoria EEPROM.

Optimizar el sistema de comunicación para permitir una mayor cantidad de efectos de sonido.